

ReturnIQ — Part 2

The Authentication System — db.js, auth-utils.js, middleware/auth.js, routes/auth.js

This part covers the four files built after Part 1: the database connection, the password/token security utilities, the middleware that protects routes, and the actual login/register/refresh endpoints. Together, these are the full working login system for ReturnIQ.

1. db.js — The Database Connection Pool

This file is the single bridge between your Node.js code and MySQL. Every other file that needs to talk to the database imports from here instead of creating its own connection.

```
const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  ...
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0
});
```

- **mysql.createPool(...)** — instead of opening one connection to MySQL and reusing just that one (slow, and it blocks if two requests come in at once), this creates a pool of up to 10 ready connections that different requests can borrow and return.
- **host / user / password / database** read from process.env — this is why the .env file matters: none of these real values are typed directly into this file.
- **|| 'localhost' and || 'root'** — these are fallback defaults. If DB_HOST isn't set in .env for some reason, it falls back to 'localhost' instead of crashing immediately with an unclear error.
- **waitForConnections: true** — if all 10 connections are busy at once, new requests wait in a queue instead of failing outright.
- **connectionLimit: 10** — caps how many simultaneous DB connections this app can open. This protects MySQL from being overwhelmed if traffic spikes.
- **queueLimit: 0** — means there's no limit on how many requests can wait in that queue (0 = unlimited waiting, rather than 0 = no waiting allowed — a common point of confusion).

```
async function query(sql, params) {
  try {
    const [results] = await pool.execute(sql, params);
    return results;
  } catch (err) {
    console.error(...);
    throw err;
  }
}
```

```
}  
}
```

This **query()** function is a small wrapper so the rest of the app never has to deal with the pool directly. **pool.execute(sql, params)** is what actually runs a SQL query.

- **params** are separate from the SQL text itself — e.g. you'd call `query('SELECT * FROM users WHERE email = ?', [email])` instead of gluing the email directly into the SQL string. This is critical: it's what prevents **SQL injection**, where a malicious user could type something like `""; DROP TABLE users; --` into a form field and have it run as real SQL if it were pasted directly into the query text. Using `?` placeholders means MySQL always treats params as pure data, never as code.
- The **try/catch** logs the error for debugging, then re-throws it — meaning whatever called `query()` still knows something failed and can handle it (e.g., sending a 500 error response), rather than the failure disappearing silently.

Interview Q: What is SQL injection, and how does your code prevent it?

Answer: SQL injection is when user input is inserted directly into a SQL command, letting an attacker sneak in their own SQL logic — for example, typing input that could delete a table or read data they shouldn't see. My `query()` function always uses parameterized queries — the `?` placeholders in the SQL string — so user input is passed separately as data, never concatenated into the SQL text itself. MySQL treats placeholder values purely as values, so even if someone typed SQL-looking text into a login field, it would just be treated as a literal (and incorrect) password string, not executed as a command.

Interview Q: Why use a connection pool instead of one single connection?

Answer: A single connection can only run one query at a time — if two people used the app simultaneously, or my AI agent made several database calls in sequence, requests would queue up and slow everything down. A pool keeps multiple ready connections open, so several queries can run concurrently without blocking each other.

2. auth-utils.js — Password & Token Security Toolkit

This file doesn't handle any web requests itself — it's a toolbox of security functions that other files (like `routes/auth.js`) call when they need to hash a password or create/check a token.

```
const SALT_ROUNDS = 10;  
  
async function hashPassword(password) {  
  return await bcrypt.hash(password, SALT_ROUNDS);  
}
```

bcrypt.hash(password, SALT_ROUNDS) scrambles the password into an irreversible string. **SALT_ROUNDS = 10** controls how many times `bcrypt` runs its scrambling algorithm internally — higher numbers make it slower to compute (which is intentional: it also makes it slower for an attacker trying to guess passwords by brute force). 10 is a widely used, sensible default — secure enough, but still fast enough not to noticeably slow down your login/signup requests.

```
async function comparePassword(plainPassword, hash) {
  return await bcrypt.compare(plainPassword, hash);
}
```

`bcrypt.compare()` doesn't "un-hash" anything (that's impossible by design) — instead, it re-runs the same scrambling process on the typed password and checks if the result matches the stored hash. Returns true or false.

```
function generateAccessToken(payload) {
  return jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '15m' });
}

function generateRefreshToken(payload) {
  return jwt.sign(payload, process.env.JWT_REFRESH_SECRET, { expiresIn: '7d' });
}
```

- **jwt.sign(payload, secret, options)** creates a token that encodes whatever data you put in *payload* (like the user's id, email, role), signs it with your secret key so it can't be forged, and sets an expiry.
- **Access token expires in 15 minutes** — short on purpose. It's sent with almost every request, so it's the more exposed token; a short lifespan limits how long a stolen one would work.
- **Refresh token expires in 7 days** — used rarely, only to get a new access token, so it can safely live longer without being sent around constantly.
- **Two different secrets** (JWT_SECRET vs JWT_REFRESH_SECRET) — if one secret were ever compromised, the other token type isn't automatically broken too.

```
function verifyToken(token, secret) {
  return jwt.verify(token, secret);
}
```

`jwt.verify()` checks the token's signature against the secret and its expiry date. If either is invalid (tampered with, wrong secret, or expired), it throws an error automatically — the calling code just needs to wrap it in `try/catch`.

Interview Q: Why does the payload include the user's role inside the JWT itself?

Answer: Putting the role in the token means the server can check permissions (is this a seller-only route?) immediately after verifying the token's signature, without needing an extra database lookup on every single request. It's a performance choice — the trade-off is that if a user's role changed, their existing token would still show the old role until it expires and they log in again.

3. middleware/auth.js — The Gatekeeper for Protected Routes

"Middleware" in Express is a function that runs *before* your actual route logic — like a security checkpoint every request passes through first. This file defines three checkpoints.

```
function requireAuth(req, res, next) {
  const authHeader = req.headers['authorization'];
  const token = authHeader && authHeader.split(' ')[1];

  if (!token) {
```

```

    return res.status(401).json({ error: '...' });
  }

  try {
    const decoded = verifyToken(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: '...' });
  }
}

```

- **req.headers['authorization']** reads the Authorization header the frontend sends, which looks like the text "Bearer eyJhbGc..." — the actual token comes after the word "Bearer".
- **authHeader.split(' ')[1]** splits that string on the space and grabs the second piece — just the token itself, dropping the word "Bearer".
- **if (!token) return res.status(401)...** — 401 means "Unauthorized." If there's no token at all, the request is rejected immediately, before even trying to verify anything.
- **req.user = decoded** — this is the key trick of middleware: it attaches the logged-in user's info onto the request object itself, so every route handler that runs afterward can just read req.user without re-checking the token.
- **next()** — tells Express "this checkpoint passed, continue to the next function" (either another middleware, or the actual route logic).
- If verifyToken() throws (invalid signature or expired), the catch block sends 401 again — same status code as "no token," so an attacker can't tell the difference between "you sent nothing" and "you sent something wrong," which is a small but real security-conscious choice.

```

function requireSeller(req, res, next) {
  requireAuth(req, res, () => {
    if (req.user.role !== 'seller') {
      return res.status(403).json({ error: '...' });
    }
    next();
  });
}

```

This is **layered middleware** — requireSeller doesn't duplicate the token-checking logic. It calls requireAuth first (passing in its own function as the "next" step), and only after that succeeds does it add one more check: is this specific user's role 'seller'? If not, it returns **403 ("Forbidden")** — different from 401. 401 means "we don't know who you are." 403 means "we know who you are, but you're not allowed here." requireCustomer works identically, just checking for 'customer' instead.

Interview Q: What's the difference between the 401 and 403 status codes in your middleware, and why does it matter?

Answer: 401 Unauthorized means the request has no valid identity at all — no token, or a broken/expired one. 403 Forbidden means the identity is valid and known, but that specific user isn't allowed to access this resource — for example, a logged-in customer trying to hit a seller-only endpoint. Distinguishing them

matters because the frontend can react differently: a 401 should trigger a redirect to login, while a 403 means the user is logged in correctly but simply doesn't have permission, which is a different UI message.

Interview Q: Why did you build `requireSeller` and `requireCustomer` on top of `requireAuth` instead of writing them as separate, standalone checks?

Answer: It avoids duplicating the token-verification logic in three places. `requireAuth` is the single source of truth for "is this token valid," and the role-specific middlewares just add one extra condition on top. If I ever needed to change how tokens are verified, I'd only update it in one place.

4. routes/auth.js — The Actual Login/Register Endpoints

This file defines the real URLs the frontend calls: `register`, `login`, `refresh`, and `me`. Each one is an Express **route handler** — a function that runs when a specific URL + HTTP method is hit.

POST /api/auth/register

```
const existing = await query('SELECT id FROM users WHERE email = ?', [email]);
if (existing.length > 0) {
  return res.status(409).json({ error: '...' });
}

const password_hash = await hashPassword(password);
const result = await query(
  'INSERT INTO users (email, display_name, role, password_hash) VALUES (?, ?, ?, ?)',
  [email, display_name, 'customer', password_hash]
);
```

- First checks if the email is already taken — **409 ("Conflict")** is the correct status code for "this already exists," more specific than a generic error.
- Password is hashed *before* being inserted — the plain-text password that arrived in the request body is never written to the database.
- The role is hardcoded as **'customer'** in the INSERT — this route can only ever create customer accounts. The seller account isn't created through self-registration at all; it was pre-seeded directly in `seed.sql`. This is a deliberate security choice: nobody can register themselves as a seller through the public API.

Interview Q: Why can't someone register as a seller through your API?

Answer: Because the register route hardcodes `role = 'customer'` in the INSERT statement — there's no code path that lets a request specify its own role. The single seller account was created directly in the database via seed data. This matches the single-seller design: EliteMart itself doesn't need a public signup flow, and blocking it entirely removes an entire category of privilege-escalation risk.

POST /api/auth/login

```
const users = await query('SELECT * FROM users WHERE email = ?', [email]);
if (users.length === 0) {
  return res.status(401).json({ error: 'Invalid email or password.' });
}
```

```

}
const isMatch = await comparePassword(password, user.password_hash);
if (!isMatch) {
  return res.status(401).json({ error: 'Invalid email or password.' });
}

```

Notice both failure cases — "email doesn't exist" and "password is wrong" — return the exact same error message and status code. This is intentional. If the app said "no account with that email" versus "wrong password" separately, an attacker could use that difference to figure out which emails have accounts at all, before even trying to guess a password. Giving one identical, vague message closes that information leak.

Interview Q: Why do you return the same error message whether the email doesn't exist or the password is wrong?

Answer: If those two cases returned different messages, an attacker could use the app to check which email addresses have registered accounts — just by seeing which emails return 'wrong password' versus 'no account found.' That's a real information leak called a user-enumeration vulnerability. Returning one identical, generic message for both cases prevents that.

```

const tokenPayload = { id: user.id, email: user.email, display_name: user.display_name, role:
user.role };
const accessToken = generateAccessToken(tokenPayload);
const refreshToken = generateRefreshToken(tokenPayload);

```

Once the password matches, both tokens are generated from the same payload data and sent back to the frontend in the response, along with basic (non-sensitive) user info for display purposes. Note that `password_hash` itself is never included anywhere in this response — only in the internal query result used to check the password.

POST /api/auth/refresh

```

const decoded = verifyToken(refreshToken, process.env.JWT_REFRESH_SECRET);
const newAccessToken = generateAccessToken({ id: decoded.id, ... });

```

This route is how a user stays logged in without re-entering their password every 15 minutes. The frontend, once its access token expires, sends the longer-lived refresh token here. If it's still valid, the server issues a brand-new access token — no database lookup needed, since all the necessary info (id, email, role) is already inside the decoded refresh token itself.

GET /api/auth/me

```

router.get('/me', requireAuth, async (req, res) => {
  const users = await query(
    'SELECT id, email, display_name, role, created_at FROM users WHERE id = ?',
    [req.user.id]
  );
  ...
});

```

Notice **requireAuth** sitting directly in the route definition, before the async handler function. This is how Express chains middleware — the request must pass through **requireAuth** first (which populates `req.user`) before this handler even runs. The SQL query also deliberately selects only specific columns (`id`, `email`, `display_name`, `role`, `created_at`) instead of `SELECT *` — this makes sure `password_hash` never accidentally gets sent back to the frontend, even by mistake.

Interview Q: Why does the `/me` route select specific columns instead of using `SELECT *`?

Answer: Using `SELECT *` would pull every column, including `password_hash` — even though I wouldn't intentionally send it to the frontend, an accidental future change (like carelessly returning the whole row) could leak it. Explicitly listing only the safe columns needed for the response is a small habit that prevents that entire class of mistake.

5. How These Four Files Work Together (The Full Login Flow)

Walking through what actually happens when someone logs in, in order:

- 1. Frontend sends `POST /api/auth/login` with email + password.
- 2. **routes/auth.js** receives it, calls **db.js**'s `query()` to look up the user by email.
- 3. It calls **auth-utils.js**'s `comparePassword()` to check the password against the stored bcrypt hash.
- 4. If it matches, **auth-utils.js**'s `generateAccessToken()` and `generateRefreshToken()` create both JWTs.
- 5. Both tokens are sent back to the frontend, which stores them and attaches the access token to every future request's Authorization header.
- 6. On any protected route (like a future "submit a return" endpoint), **middleware/auth.js**'s `requireAuth` runs first, calls **auth-utils.js**'s `verifyToken()` to check the access token, and only lets the request through if it's valid — attaching the decoded user info to `req.user` for the route to use.

This is a clean separation of responsibilities: **db.js** only knows about the database, **auth-utils.js** only knows about security math (hashing/tokens), **middleware/auth.js** only knows about checking requests, and **routes/auth.js** coordinates all three to actually handle a login. This kind of separation is exactly what makes each piece easy to explain individually in an interview.

End of Part 2 — send the next batch of files (Phase 3 customer routes) for Part 3.